Tutorial 2

RECURSIVE BEST FIRST SEARCH

reccurence/repetition

- recursive algorithm that mimics the operation of best-first search

- uses f-limit value to keep track of the f-value of best alternative path from any ancestor to current node

- if current node exceeds the limit – recursion goes back to altenative paths

- RBFS replaces f-value of each node with a backed-up value (best value)

$f(n) = g(n) + h(n)$

estimated cost $s.f = \max(s.g + s.h, node.f)$ → heuristic

cost

Best = best successor node → smallest f value

Best.f > f_limit return failure

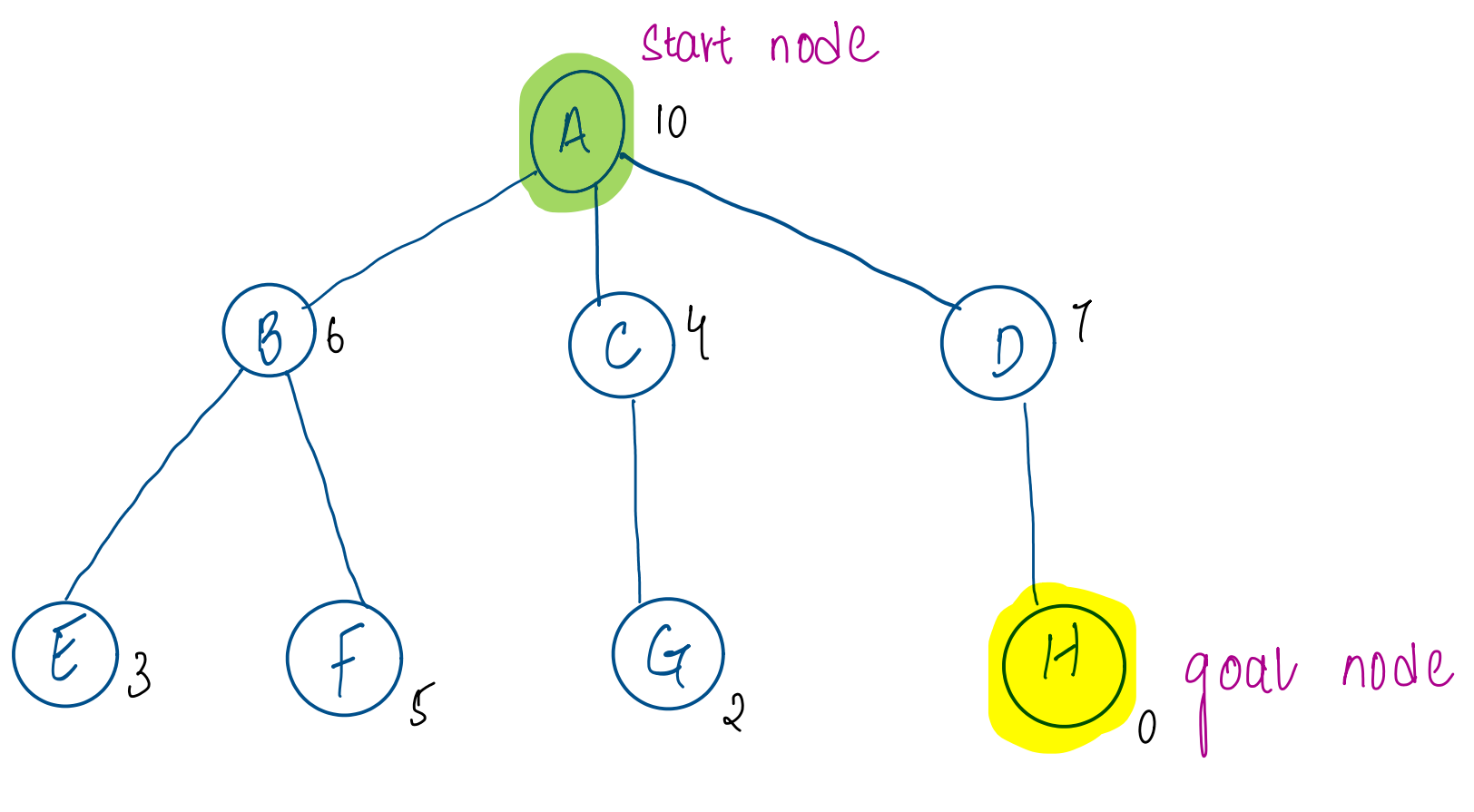
Alt = second best succ node

f_limit = min(f_limit, alt.f)

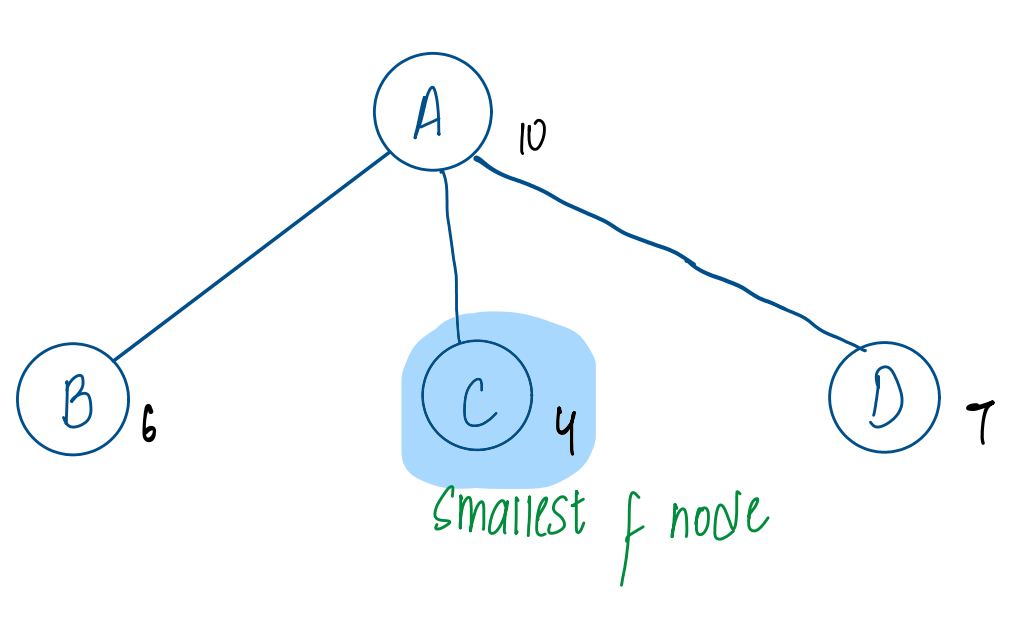
Algorithm

① start from initial node
② expand initial node to generate all of its children
③ Pick the child with the smallest f(n) and move to it.
④ if new selected node is goal – return solution
⑤ if node has no children – back track
⑥ if node children all exceed current f-value – back track
⑦ update the f-value to next best value for alternative better search.
⑧ recursive search till goal found

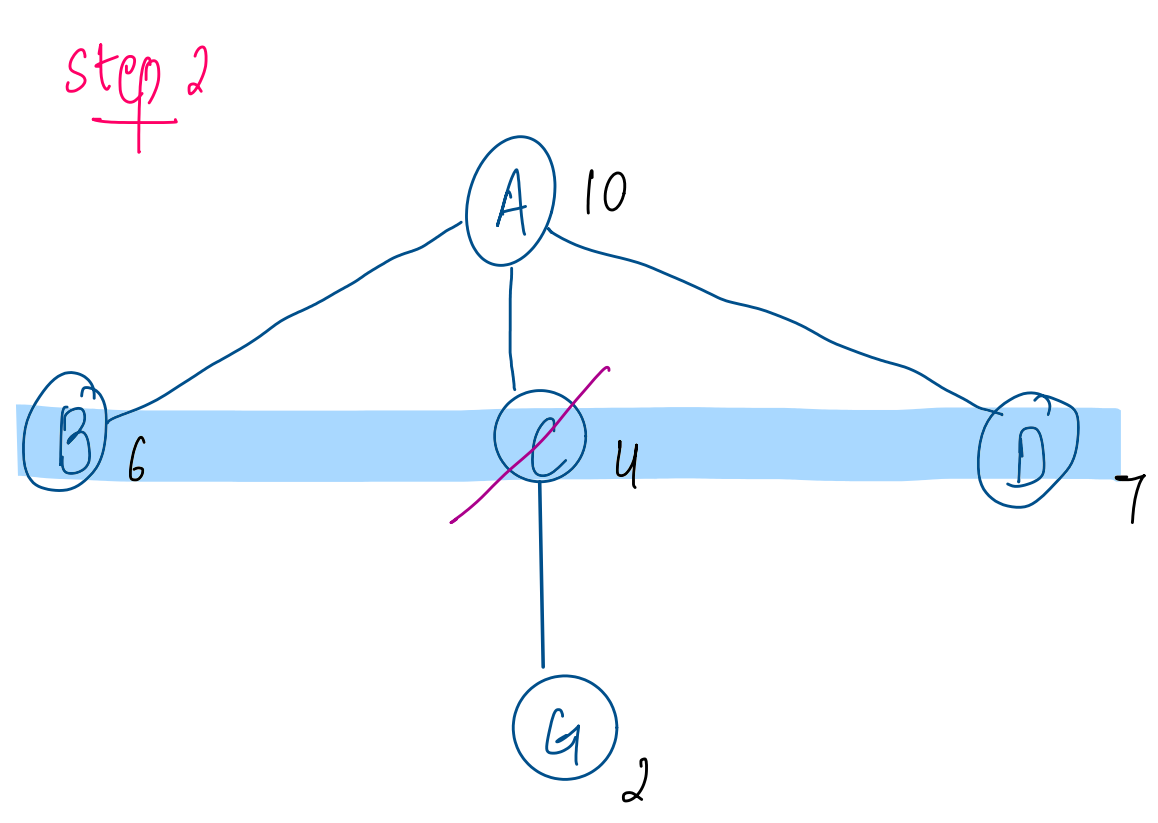
Example 1


Start node
A 10
B 6
C 4
D 7
E 3
F 5
G 2
H 0
goal node

Step 1

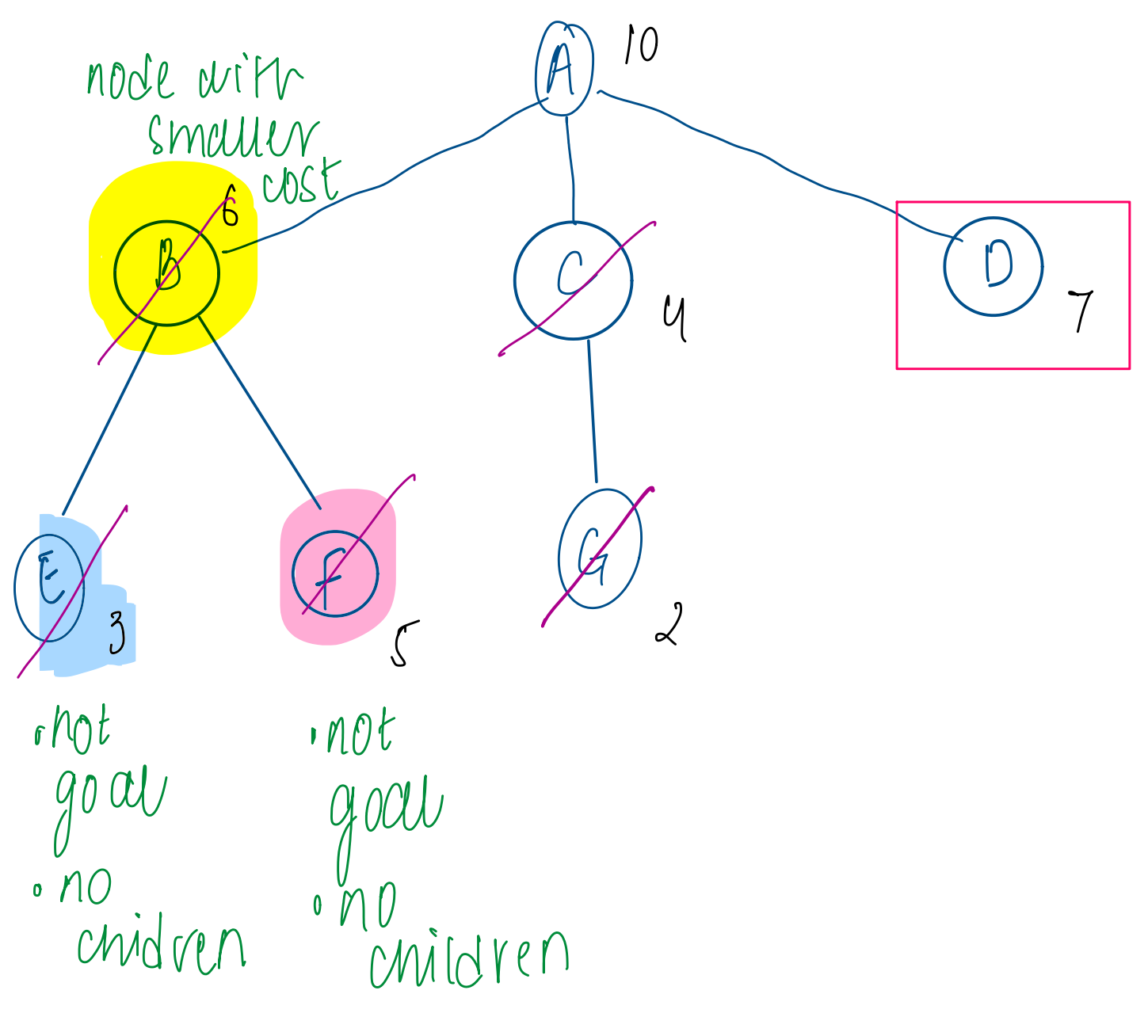

A 10
B 6
C 4
D 7
smallest f node

Step 2

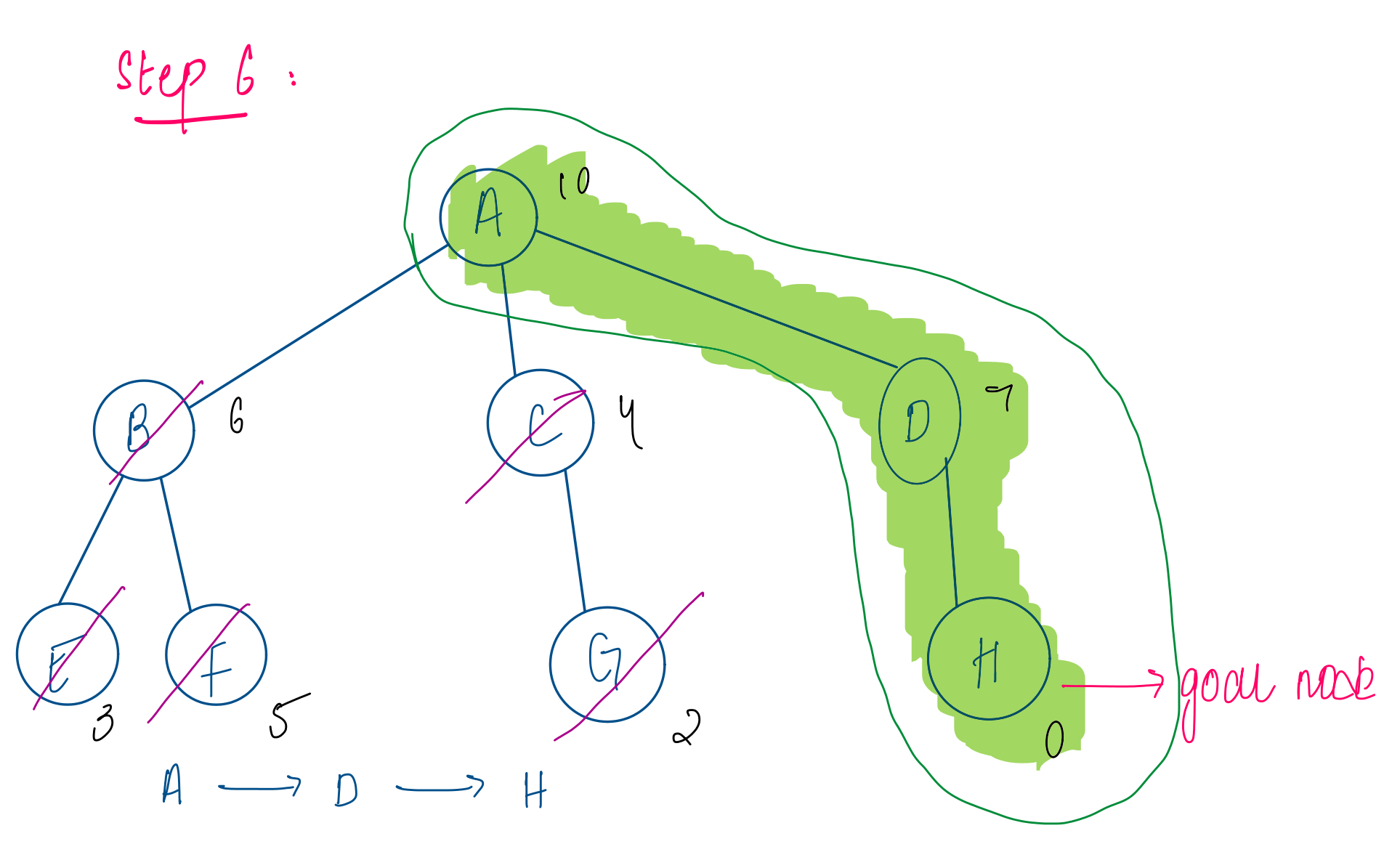

A 10
B 6
C 4
D 7
G 2

- no children
- no smaller value on previous nodes
- terminate

Step 3 + 4 + 5

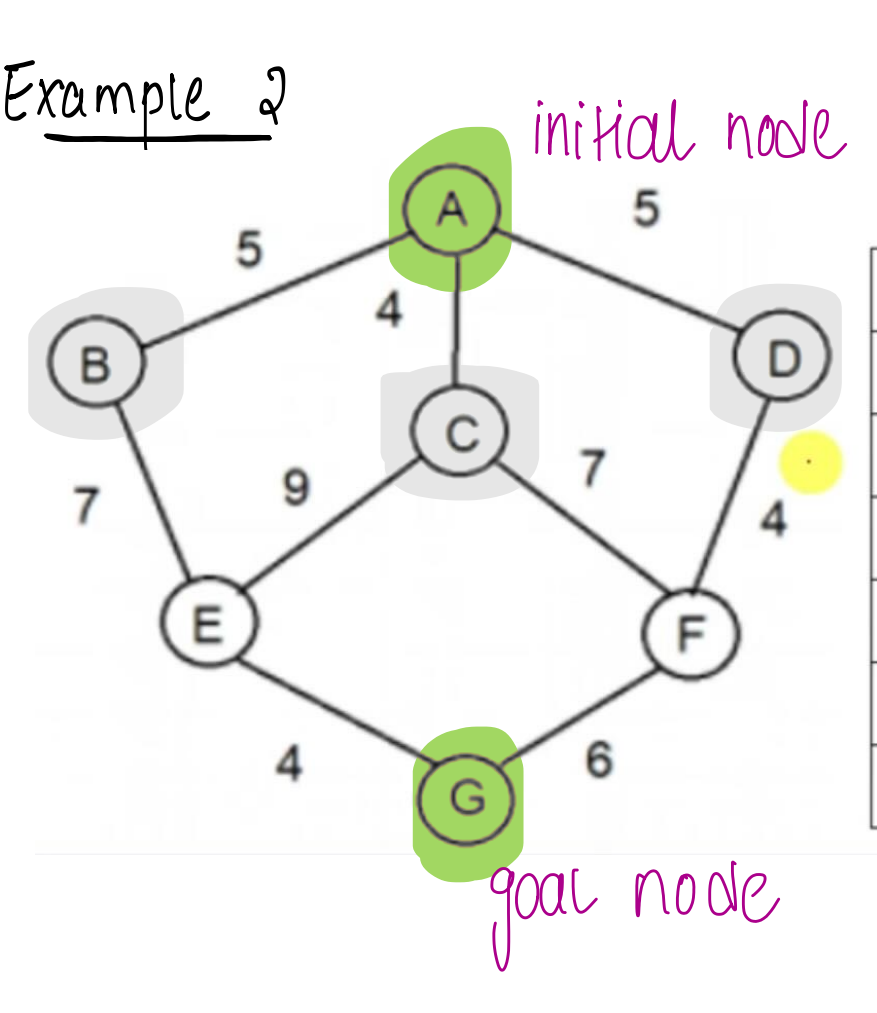

node with smaller cost
A 10
B 6
C 4
D 7
E 3
F 5
G 2
• not goal
• no children
• not goal
• no children

Step 6 :


A 10
B 6
C 4
D 7
E 3
F 5
G 2
H 0
goal node

A → D → H

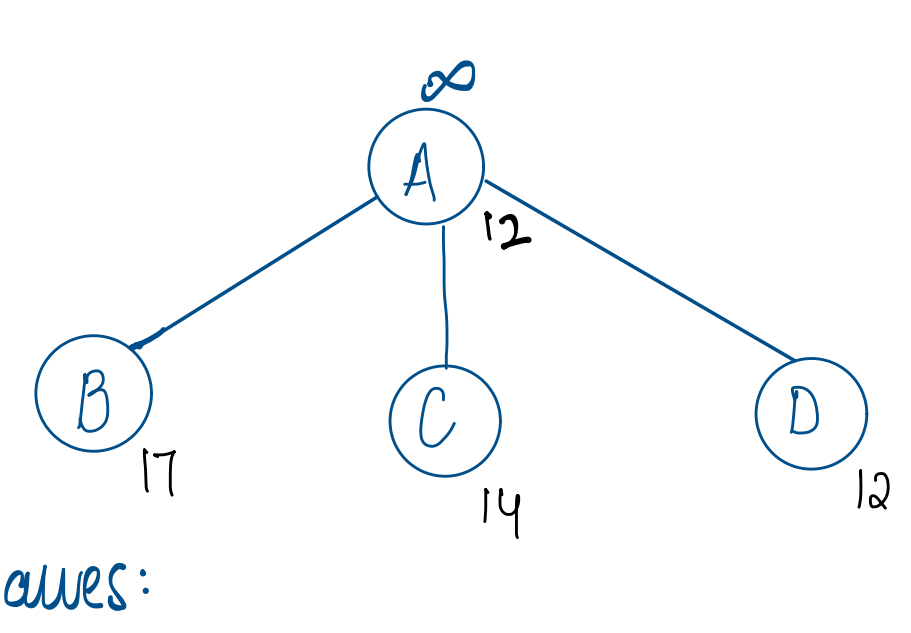
Example 2


initial node
A
B
C
D
E
F
G
5
5
4
7
9
7
4
4
6
goal node

A	12
B	12
C	10
D	7
E	2
F	6
G	0

$s.f = \max(s.g + s.h, node.f)$

Step 1


∞
A 12
B 17
C 14
D 12

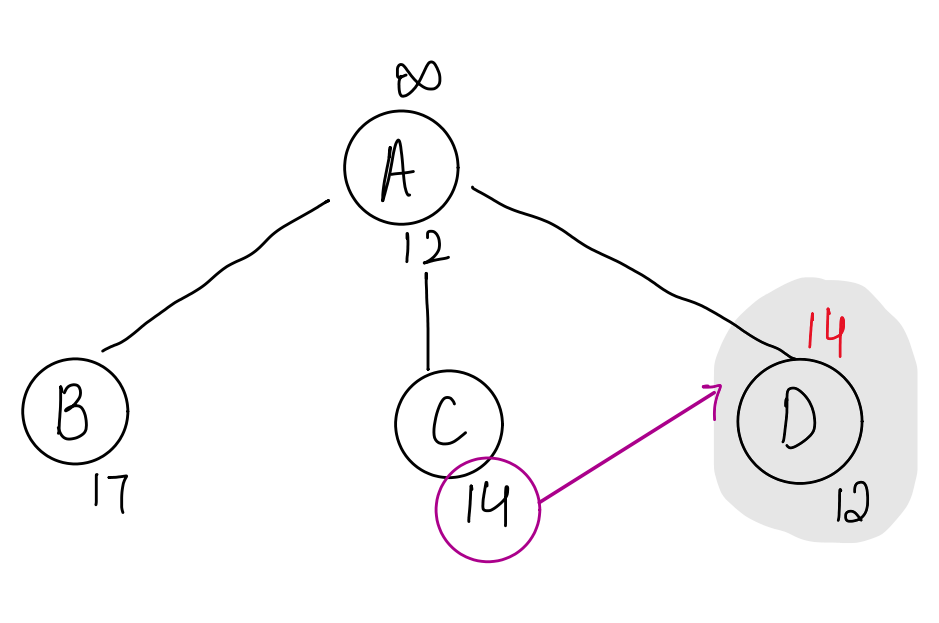
① f values:
A = 0 + 12 = 12
B = 5 + 12 = 17
s.f = max(s.g + s.h, node.f)
max(17, 12) = 17
C = 4 + 10 = 14
s.f = max(14, 12) = 14
D = 5 + 7 = 12
s.f = max(12, 12) = 12

② Best successor = lowest cost
D = 12

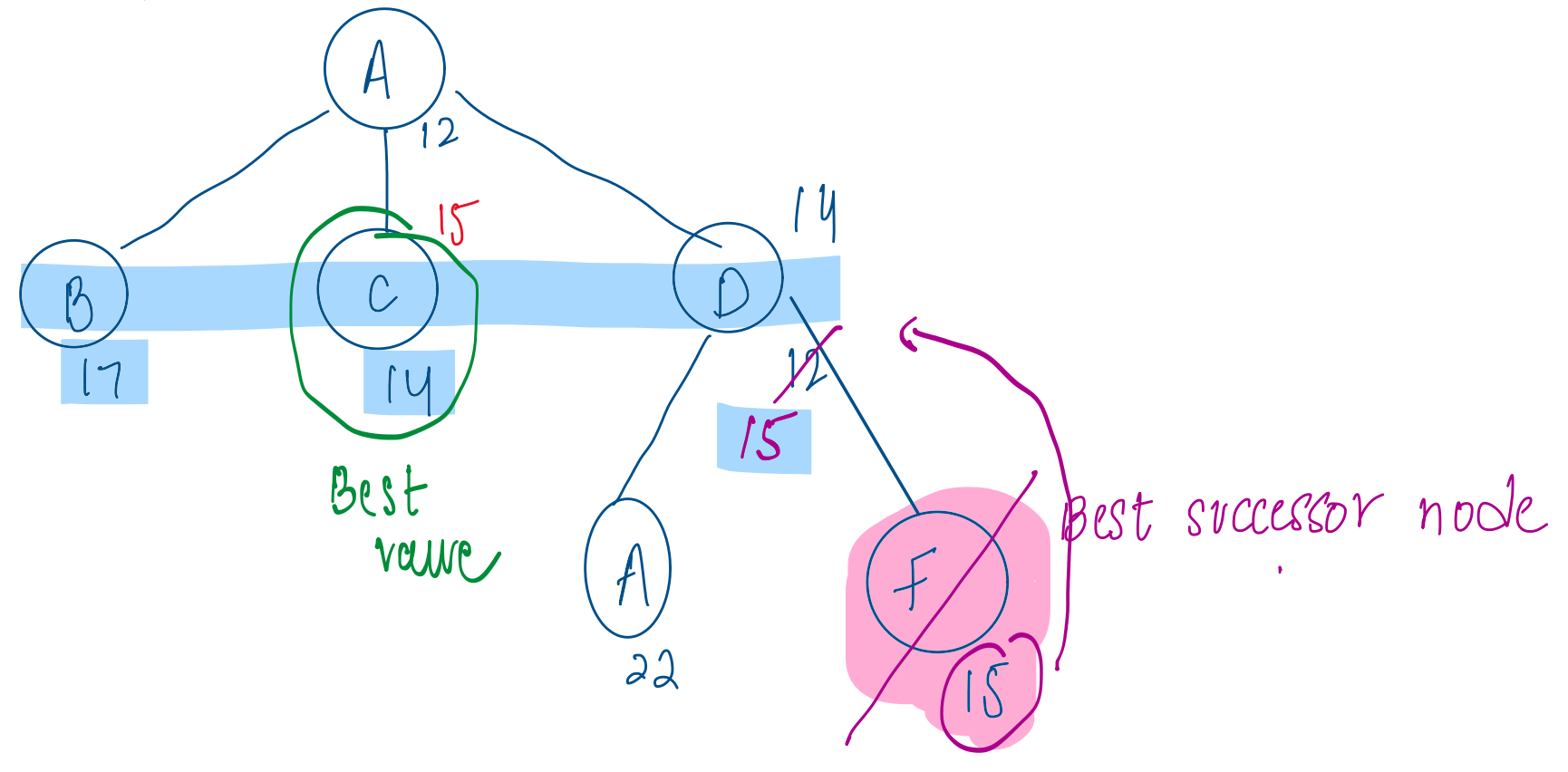
③ compare
12 > ∞
not true

④ 2nd best successor node
C = 14

⑤ new f-limit
f-limit = min(f-limit, alt.f)
= min(∞, 14)
= 14


∞
A 12
B 17
C 14
D 14 12

Step 2


∞
A 12
B 17
C 15 14
Best value
D 14 12 15
A 22
F 15
Best successor node

f-values:

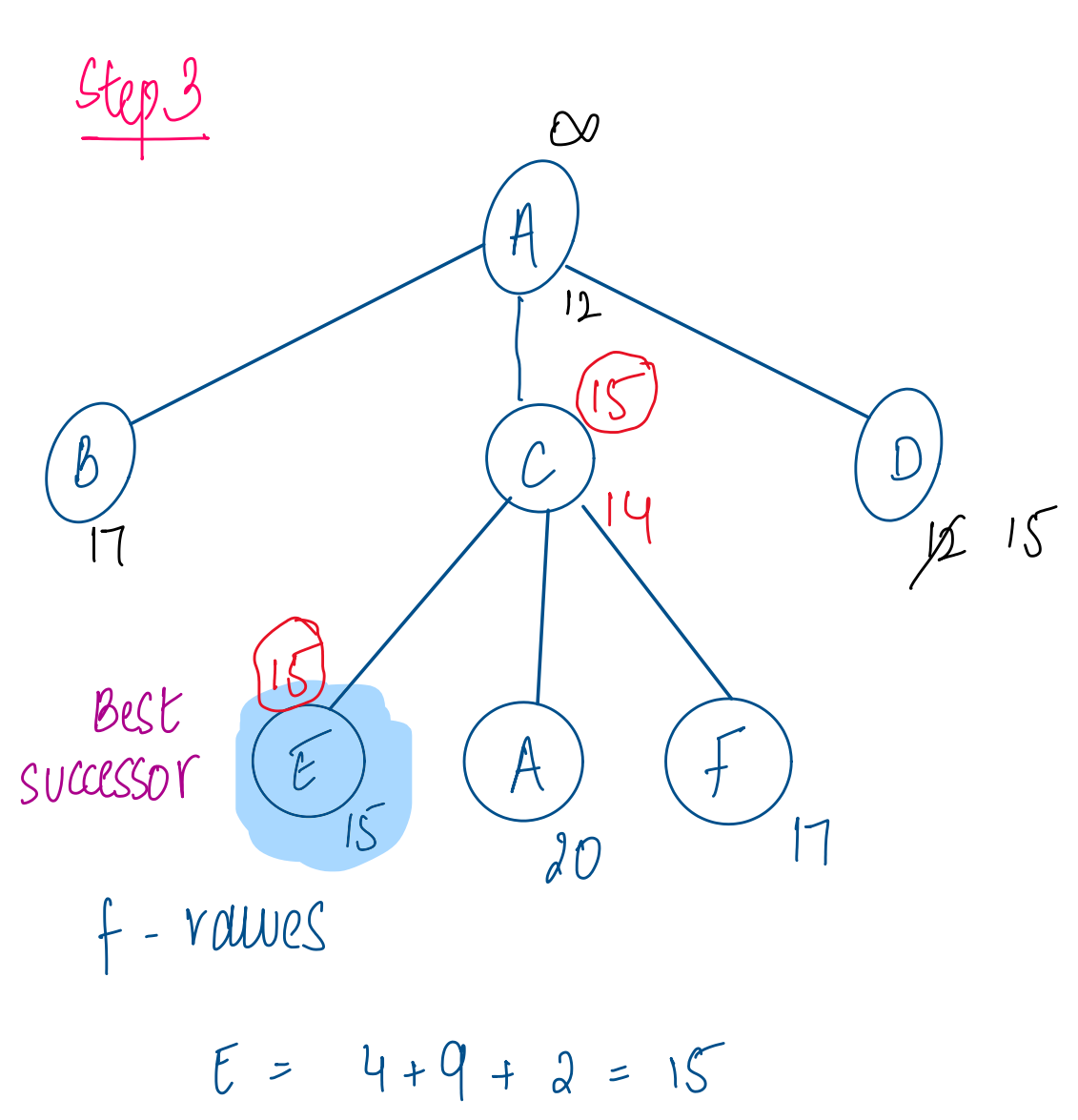
A = 5 + 12 + 5 = 22
= max(22, 12)
= 22

F = 5 + 4 + 6 = 15
= max(15, 12)
= 15

Best f > f limit return failure
15 > 14
failure

14 > ∞ : no
so explore
next f-limit = 15

Step 3


∞
A 12
B 17
C 15 14
D 12 15
Best successor
E 16 15
A 20
F 17

f-values

E = 4 + 9 + 2 = 15
= max(15, 15)
= 15

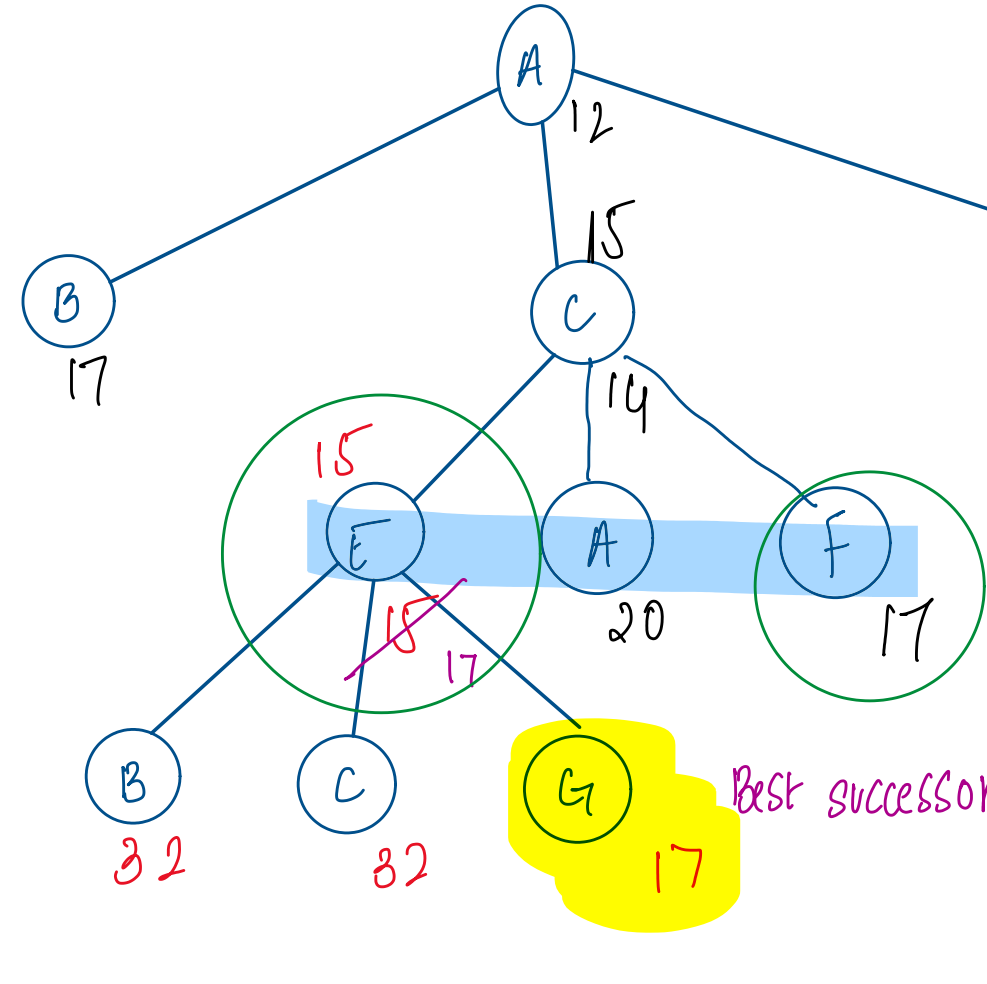
A = 4 + 4 + 12 = 20
= max(20, 15)
= 20

F = 4 + 7 + 6 = 17
= max(17, 15)
= 17

Best. f > f-limit return failure
15 > 15
failure

Alternate = F
f-limit = min(f-limit, alt.f)
= min(15, 17)
= 15

Step 4


∞
A 12
B 17
C 15 14
D 15
E 15 17
A 20
F 17
B 32
C 32
G 17
Best successor

f-value:

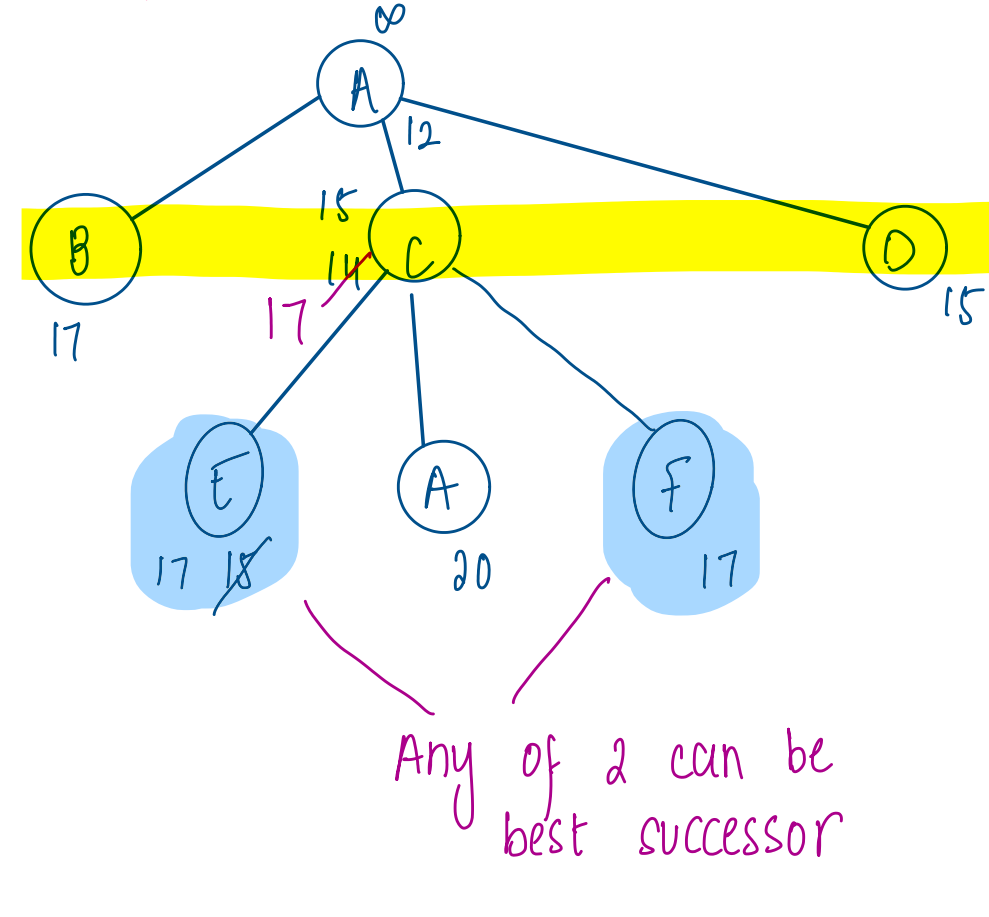
B = 4 + 9 + 7 + 12 = 32
= max(32, 15)
= 32

C = 4 + 9 + 9 + 10 = 32
= max(32, 15)
= 32

G = 4 + 9 + 4 + 0 = 17
= max(17, 15)
= 17

Best. f > f-limit return failure
17 > 15
failure
replace 15 with 17

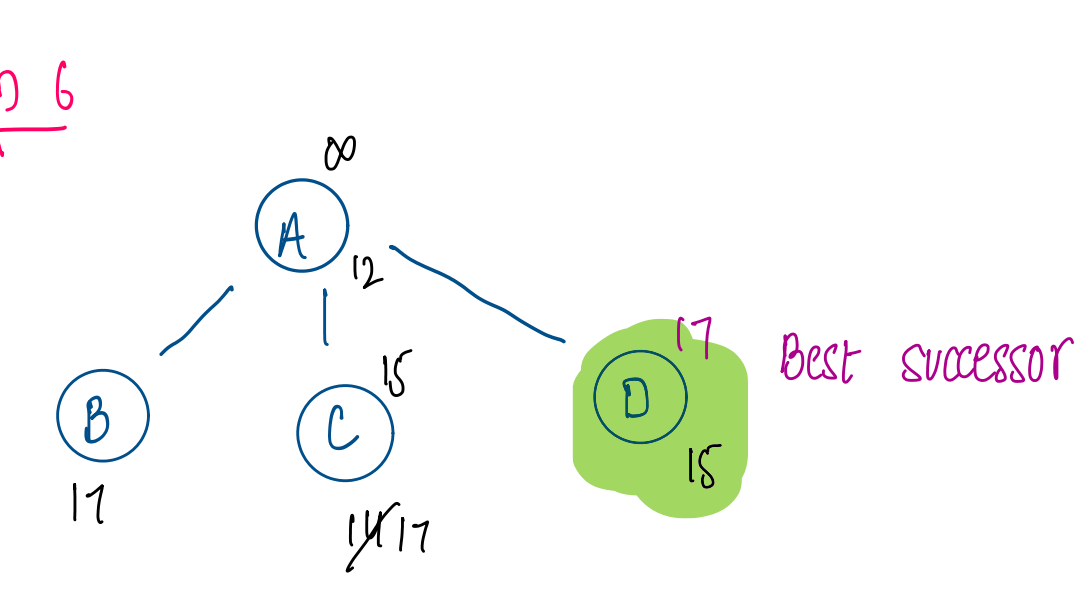
Step 5


∞
A 12
B 17
C 15 14 17
D 15
E 17 15
A 20
F 17
Any of 2 can be best successor

lets proceed with E :

Best. f > f-limit return failure
17 > 15
failure
replace 14 with 17

Step 6


∞
A 12
B 17
C 15 14 17
D 17 15
Best successor

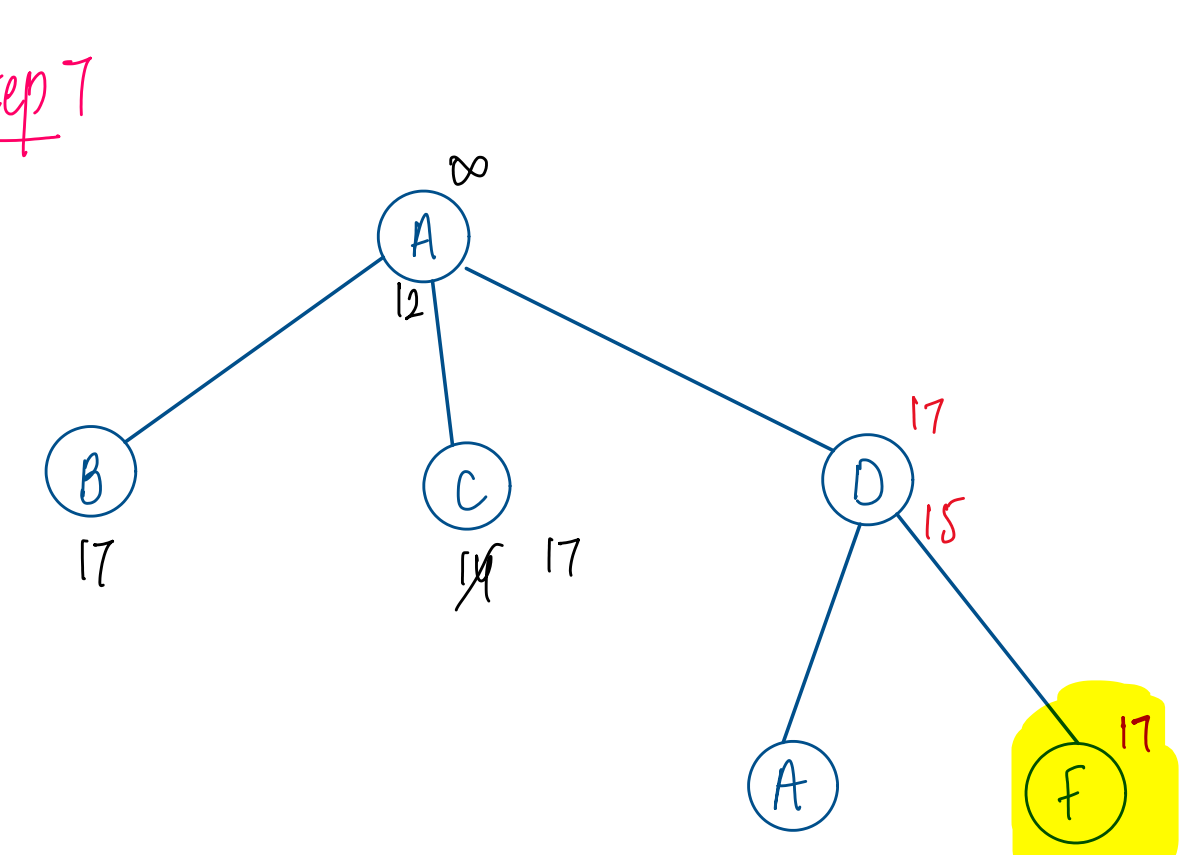
Best. f > f-limit return failure
15 > ∞
failure

Alternative
C and B can both be taken as equal value

Taking C :
f-limit = min(f-limit, alt.f)
= min(∞, 17)
= 17

limit of D = 17

Step 7


∞
A 12
B 17
C 14 17
D 17 15
A 22
F 17 15

f-values:

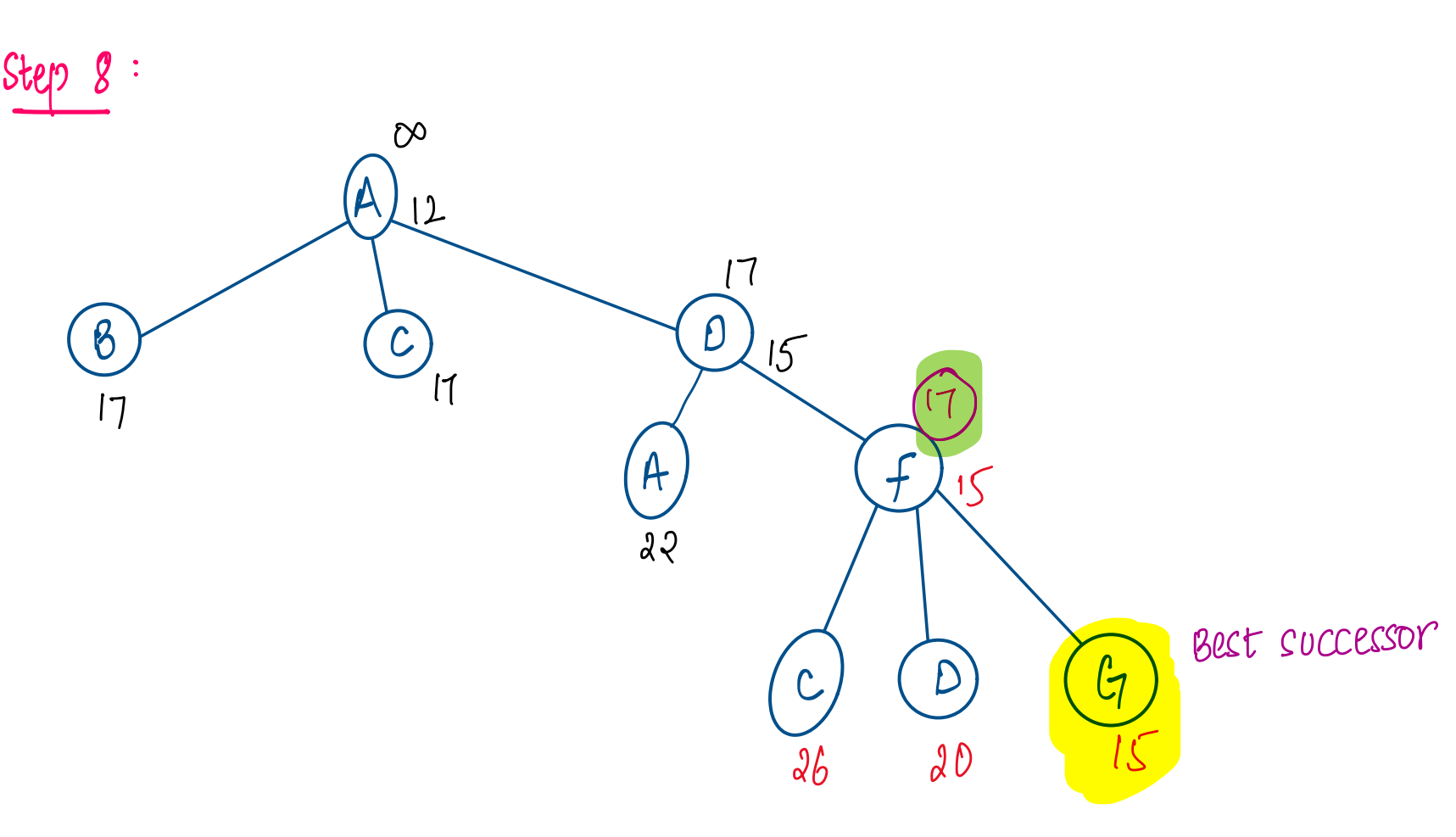
A = 5 + 5 + 12 = 22
= max(22, 15)
= 22

F = 5 + 4 + 2 = 11
= max(11, 15)
= 15

Best - f > f-limit return failure
15 > 17 NO

Alternative = A
f-limit = min(f-limit, alt.f)
= min(17, 22)
= 17

Step 8 :


∞
A 12
B 17
C 17
D 17 15
A 22
F 17 15
C 26
D 20
G 15
Best successor

f-values:

C = max(5 + 4 + 7 + 10, 15)
= 26
D = max(5 + 4 + 4 + 7, 15)
= 20
G = max(5 + 4 + 6, 15)
= 15

Best. f > f limit
15 > 17

Alternate = D

f-limit = min(17, 20)
= 17

G is goal node.

A → D → F → G
5 + 4 + 6 = 15

matches node cost